

Network Traffic Analysis Report

Author: Benedict Kam

Student Code: S5

Trainer Name: Samson

Table of Contents

The report includes an introduction, methodologies, discussion, conclusion, recommendations, and references.

Introduction

This report focuses on analyzing network traffic captured in a pcap file during the execution of a network reconnaissance script. The script was used to connect to a remote server via SSH and execute an nmap scan and whois lookup on a target domain. The primary objective of this report is to evaluate the security implications of the SSH protocol and examine its behavior using packet analysis. The analysis will also compare SSH with its insecure counterparts and discuss methods for enhancing network security.

Methodologies

To analyze the network traffic, a tcpdump capture was performed on the remote server before executing the script on the local machine, generating a pcap file containing all network interactions. The pcap file was analyzed using Wireshark to inspect SSH traffic, identify key packet exchanges, and assess encryption methods. SSH was compared with insecure protocols such as Telnet, HTTP, and FTP to highlight security advantages. The impact of SSH on confidentiality, integrity, and availability was evaluated.

Discussion

SSH is designed to provide encrypted remote access to servers and is widely used for secure command execution, file transfers, and tunneling. Its key features include strong authentication mechanisms, data encryption, and protection against eavesdropping. The protocol operates using a client-server model where the client initiates a connection to the server on port 22. The server responds with a public key for encryption negotiation, and the client verifies the server's identity before establishing an encrypted session. User authentication occurs through password-based or key-based authentication, after which the client can execute commands securely.

1	0.000000	213.144.135.21	192.168.182.129	TLSv1.2	590 Application Data
2	0.000040	192.168.182.129	213.144.135.21	TCP	54 57394 → 9001 [ACK] Seq=1 Ack=537 Win=65535 Len=0
3	0.311020	192.168.182.129	192.168.182.129	TCP	74 39292 → 22 [SYN] Seq=0 Win=32128 Len=0 MSS=1460 SACK_PERM TSval=1245567908 TSecr=0 WS=128
4	0.311064	192.168.182.129	192.168.182.129	TCP	74 22 → 39292 [SYN, ACK] Seq=0 Ack=1 Win=65108 Len=0 MSS=1460 SACK_PERM TSval=2440042057 TSecr=1245567908 WS=128
5	0.312215	192.168.182.128	192.168.182.129	TCP	66 39292 → 22 [ACK] Seq=1 Ack=1 Win=32128 Len=0 TSval=1245567909 TSecr=2440042057
6	0.312961	192.168.182.128	192.168.182.129	SSHv2	98 Client: Protocol (SSH-2.0-OpenSSH_9.7p1 Debian-7)
7	0.312980	192.168.182.129	192.168.182.128	TCP	66 22 → 39292 [ACK] Seq=1 Ack=33 Win=65152 Len=0 TSval=2440042059 TSecr=1245567910
8	0.320046	192.168.182.129	192.168.182.128	SSHv2	168 Server: Protocol (SSH-2.0-OpenSSH_8.9p1 Ubuntu-Subunt0.10)
9	0.320376	192.168.182.128	192.168.182.129	TCP	66 39292 → 22 [ACK] Seq=33 Ack=43 Win=32128 Len=0 TSval=1245567917 TSecr=2440042066
10	0.321170	192.168.182.128	192.168.182.129	TCP	1514 39292 → 22 [ACK] Seq=33 Ack=43 Win=32128 Len=1448 TSval=1245567918 TSecr=2440042066 [TCP segment of a reassembled PDU]
11	0.321177	192.168.182.128	192.168.182.129	SSHv2	154 Client: Key Exchange Init
12	0.321632	192.168.182.129	192.168.182.128	SSHv2	1178 Server: Key Exchange Init
13	0.367226	192.168.182.128	192.168.182.129	TCP	66 39292 → 22 [ACK] Seq=1569 Ack=1155 Win=31872 Len=0 TSval=1245567964 TSecr=2440042067
14	0.389547	192.168.182.128	192.168.182.129	SSHv2	1274 Client: Diffie-Hellman Key Exchange Init
15	0.389931	192.168.182.129	192.168.182.128	SSHv2	1514 Server: Diffie-Hellman Key Exchange Reply, New Keys
16	0.390965	192.168.182.129	192.168.182.128	SSHv2	182 Server:
17	0.391321	192.168.182.128	192.168.182.129	TCP	66 39292 → 22 [ACK] Seq=2777 Ack=2719 Win=31872 Len=0 TSval=1245567988 TSecr=2440042137
18	0.411337	192.168.182.128	192.168.182.129	SSHv2	82 Client: New Keys
19	0.455301	192.168.182.129	192.168.182.128	TCP	66 22 → 39292 [ACK] Seq=2719 Ack=2793 Win=64128 Len=0 TSval=2440042201 TSecr=1245568008
20	0.455703	192.168.182.128	192.168.182.129	SSHv2	110 Client:
21	0.455717	192.168.182.129	192.168.182.128	TCP	66 22 → 39292 [ACK] Seq=2719 Ack=2837 Win=64128 Len=0 TSval=2440042201 TSecr=1245568052
22	0.455928	192.168.182.129	192.168.182.128	SSHv2	110 Server:
23	0.456671	192.168.182.128	192.168.182.129	SSHv2	126 Client:
24	0.466383	192.168.182.129	192.168.182.128	SSHv2	118 Server:
25	0.471763	192.168.182.129	192.168.182.128	SSHv2	159 Client:
26	0.479911	192.168.182.129	192.168.182.128	SSHv2	94 Server:
27	0.480642	192.168.182.128	192.168.182.129	SSHv2	178 Client:

The pcap analysis shows a typical SSH handshake process. The first few packets illustrate a TCP three-way handshake between the client and server, establishing a reliable connection. The client sends a SYN packet to initiate communication, which the server acknowledges with a SYN-ACK, followed by an ACK from the client. After the handshake, the SSH protocol negotiation begins. The server and client exchange protocol versions, and the key exchange process starts with both parties sending key exchange initialization messages. Diffie-Hellman key exchange is then used to securely derive a shared secret, ensuring encrypted communication. Once the encryption is established, authentication occurs, followed by the secure execution of commands.

100	1.686190	192.168.182.129	192.168.182.128	SSHv2	150 Server:
101	1.686837	192.168.182.128	192.168.182.129	TCP	66 39292 → 22 [ACK] Seq=4669 Ack=7387 Win=31872 Len=0 TSval=1245569283 TSecr=2440043432
102	2.128473	192.168.182.128	192.168.182.129	SSHv2	102 Server:
103	2.129529	192.168.182.129	192.168.182.128	SSHv2	214 Server:
104	2.129705	192.168.182.129	192.168.182.128	SSHv2	238 Server:
105	2.129829	192.168.182.128	192.168.182.129	TCP	66 39292 → 22 [ACK] Seq=4669 Ack=7423 Win=31872 Len=0 TSval=1245569726 TSecr=2440043875
106	2.129841	192.168.182.128	192.168.182.129	TCP	66 39292 → 22 [ACK] Seq=4669 Ack=7571 Win=31872 Len=0 TSval=1245569727 TSecr=2440043875
107	2.129843	192.168.182.128	192.168.182.129	TCP	66 39292 → 22 [ACK] Seq=4669 Ack=7735 Win=31872 Len=0 TSval=1245569727 TSecr=2440043875
108	2.756258	192.168.182.128	89.190.6.9	TLSv1.2	590 Application Data
109	2.756263	89.190.6.9	192.168.182.128	TCP	66 9030 → 58754 [ACK] Seq=1 Ack=537 Win=64240 Len=0
110	3.168095	192.168.182.129	213.144.135.21	TLSv1.2	590 Application Data
111	3.169141	213.144.135.21	192.168.182.129	TCP	60 9001 → 57394 [ACK] Seq=537 Ack=537 Win=64240 Len=0
112	3.712040	192.168.182.129	192.168.182.128	SSHv2	150 Server:
113	3.712444	192.168.182.128	192.168.182.129	TCP	66 39292 → 22 [ACK] Seq=4669 Ack=7819 Win=31872 Len=0 TSval=1245571309 TSecr=2440045458
114	3.749366	192.168.182.129	192.168.182.128	SSHv2	126 Server:
115	3.749779	192.168.182.129	192.168.182.128	TCP	66 39292 → 22 [ACK] Seq=4669 Ack=7879 Win=31872 Len=0 TSval=1245571346 TSecr=2440045495
116	3.758093	192.168.182.128	192.168.182.129	SSHv2	102 Server:
117	3.758454	192.168.182.129	192.168.182.128	TCP	66 39292 → 22 [ACK] Seq=4669 Ack=7915 Win=31872 Len=0 TSval=1245571355 TSecr=2440045504
118	3.762233	192.168.182.129	192.168.182.128	SSHv2	134 Server:
119	3.762617	192.168.182.128	192.168.182.129	TCP	66 39292 → 22 [ACK] Seq=4669 Ack=7983 Win=31872 Len=0 TSval=1245571359 TSecr=2440045508
120	3.974660	192.168.182.129	192.168.182.128	SSHv2	134 Server:
121	3.975111	192.168.182.128	192.168.182.129	TCP	66 39292 → 22 [ACK] Seq=4669 Ack=8051 Win=31872 Len=0 TSval=1245571572 TSecr=2440045720

The second referenced Wireshark screenshot further illustrates encrypted TLSv1.2 communication to an external host, suggesting a secure exchange with a web service. This supports the assertion that encryption protocols like SSH and TLS play a vital role in maintaining network security and preventing eavesdropping attacks.

SSH ensures confidentiality by encrypting all transmitted data, making interception by attackers ineffective. It guarantees integrity through cryptographic checks that prevent data tampering during transmission. Availability is maintained through reliable remote access, although strict security controls may block unauthorized connections. Compared to insecure protocols such as Telnet and FTP, which transmit credentials and data in plaintext, SSH encrypts communication, making it resistant to packet sniffing and man-in-the-middle attacks.

Secure Network Protocol Implementation

To further demonstrate secure communications, a basic SSH client-server application was developed. This application uses key-based authentication to establish an encrypted session, ensuring secure data exchange. Through this implementation, it is demonstrated how SSH addresses concerns related to insecure protocols. By enforcing encryption and strong authentication, SSH effectively protects network communications from unauthorized access and data tampering.

Conclusion

The analysis of the captured network traffic confirms that SSH provides a secure and efficient means of remote access. Its encrypted communication model mitigates risks associated with plaintext protocols such as Telnet and FTP. By implementing SSH alongside best security practices, organizations can enhance their cybersecurity posture and safeguard sensitive data transmissions.

Recommendations

To further strengthen network security, it is important to enforce key-based authentication instead of passwords to enhance authentication security. Disabling root login is necessary to restrict direct SSH access to privileged accounts and prevent unauthorized use. Implementing firewall rules to limit SSH access to trusted IP addresses reduces exposure to attacks. Regularly analyzing SSH logs and network traffic helps in detecting potential security threats. Updating encryption algorithms and using strong ciphers such as AES-256 ensures that SSH configurations remain up to date and secure.

References

SSH Protocol Documentation

- OpenSSH: <https://www.openssh.com/manual.html>
- RFC 4251 (SSH Protocol Architecture): <https://datatracker.ietf.org/doc/html/rfc4251>

Wireshark Documentation

- Wireshark User Guide: https://www.wireshark.org/docs/wsug_html_chunked/
- SSH Traffic Analysis with Wireshark: <https://wiki.wireshark.org/SSH>

Network Security Best Practices

- NIST Guidelines on Secure Shell (SSH):
<https://csrc.nist.gov/publications/detail/sp/800-189/final>
- OWASP Secure Configuration of SSH:
https://cheatsheetseries.owasp.org/cheatsheets/Secure_SSH_Practices_Cheat_Sheet.html

Tcpdump and Network Analysis

- Tcpdump Manual: <https://www.tcpdump.org/manpages/tcpdump.1.html>
- Introduction to Packet Analysis: <https://www.sans.org/white-papers/packet-analysis/>

TLSv1.2 and Secure Communication

- TLS 1.2 RFC: <https://datatracker.ietf.org/doc/html/rfc5246>
- TLS Traffic Analysis with Wireshark: <https://wiki.wireshark.org/TLS>